Convolutional Neural Networks (CNN)

Previous session

In the previous session you have:

- Understood the **inspiration of CNNs from the human visual system** — learning hierarchically from edges → shapes → objects.
- Studied **the key building blocks of CNNs.**
- Understood **the Convolutional Layer** — how filters (kernels) slide over input images to produce the feature maps.
  - Explored **mathematics of 2-D convolution**, effect of kernel size, padding, and stride on output dimensions.

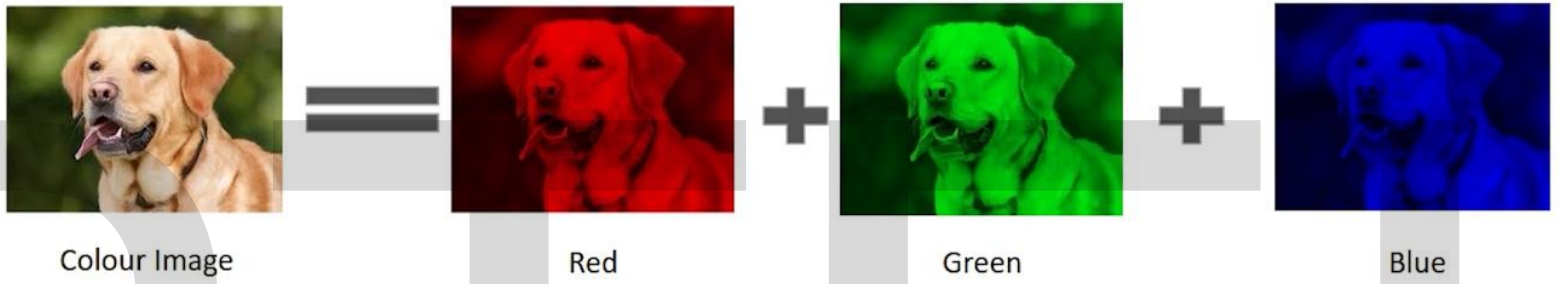
Discussed **Valid vs Same padding** and the **need for preserving border information**.

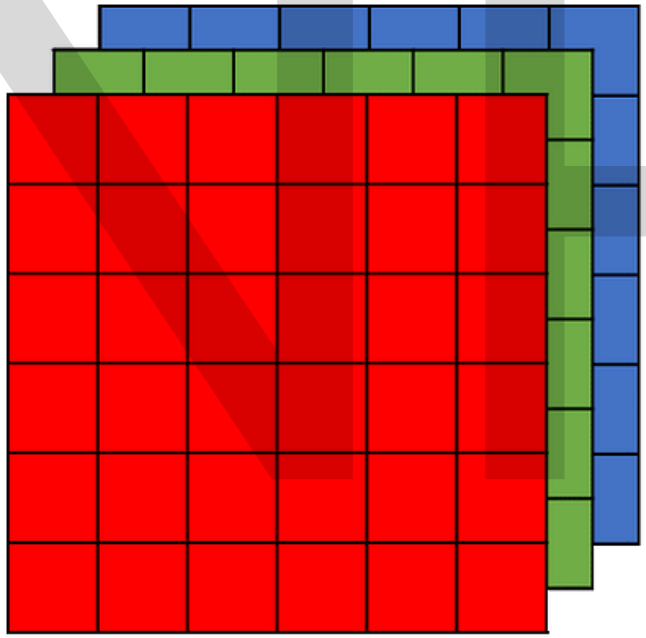
Demonstrated **how stride affects feature map size.**

Today's Session Overview

- Understand how convolutions work for RGB images with three channels.
- Learn about further layers of a CNN:
  1. **Activation Layer (ReLU)** — introduces non-linearity.
  2. **Pooling Layer** — reduces spatial dimensions and controls overfitting.
  3. **Flatten Layer** — converts 2-D feature maps to 1-D vectors for classification.
- Observe how these layers connect to fully connected (FC) layers in deep CNN architectures.

Convolution operation on RGB Images



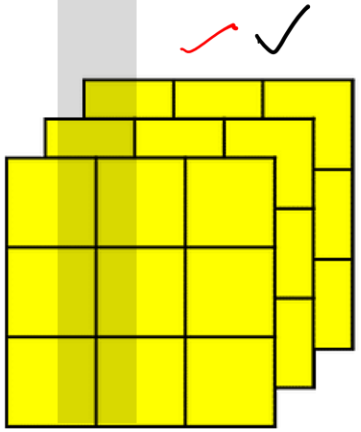


$6 \times 6 \times 3$ ← Channels

Height

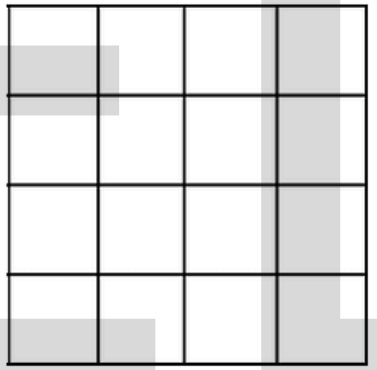
Width

*



$3 \times 3 \times 3$

=



Output in general: 4×4

$$(n - f + 1) \times (n - f + 1)$$

= (6 – 3 + 1) × (6 – 3 + 1)

= 4 × 4

9	0	6	6	9
9	9	7	3	2
8	0	9	0	5
1	1	3	0	2
5	1	7	9	3

*

1	0	-1
1	1	-1
1	1	-1

13	16	9
0	19	10
-3	3	18

9	4	3	0	3
5	1	7	3	4
2	0	1	9	7
1	8	1	0	8
0	4	3	6	5

*

1	0	1
-1	0	1
-1	0	-1

11	-3	-5
9	5	8
0	-9	7

5	6	3	4	6
1	5	8	2	4
8	3	2	3	5
7	1	5	9	5
0	1	3	9	9

*

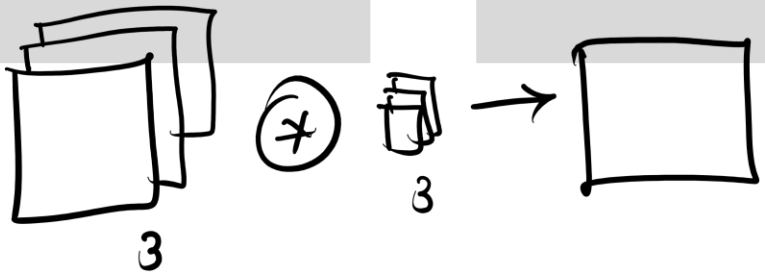
1	-1	-1
-1	0	1
-1	0	1

-3	-4	-8
-20	3	5
4	14	0

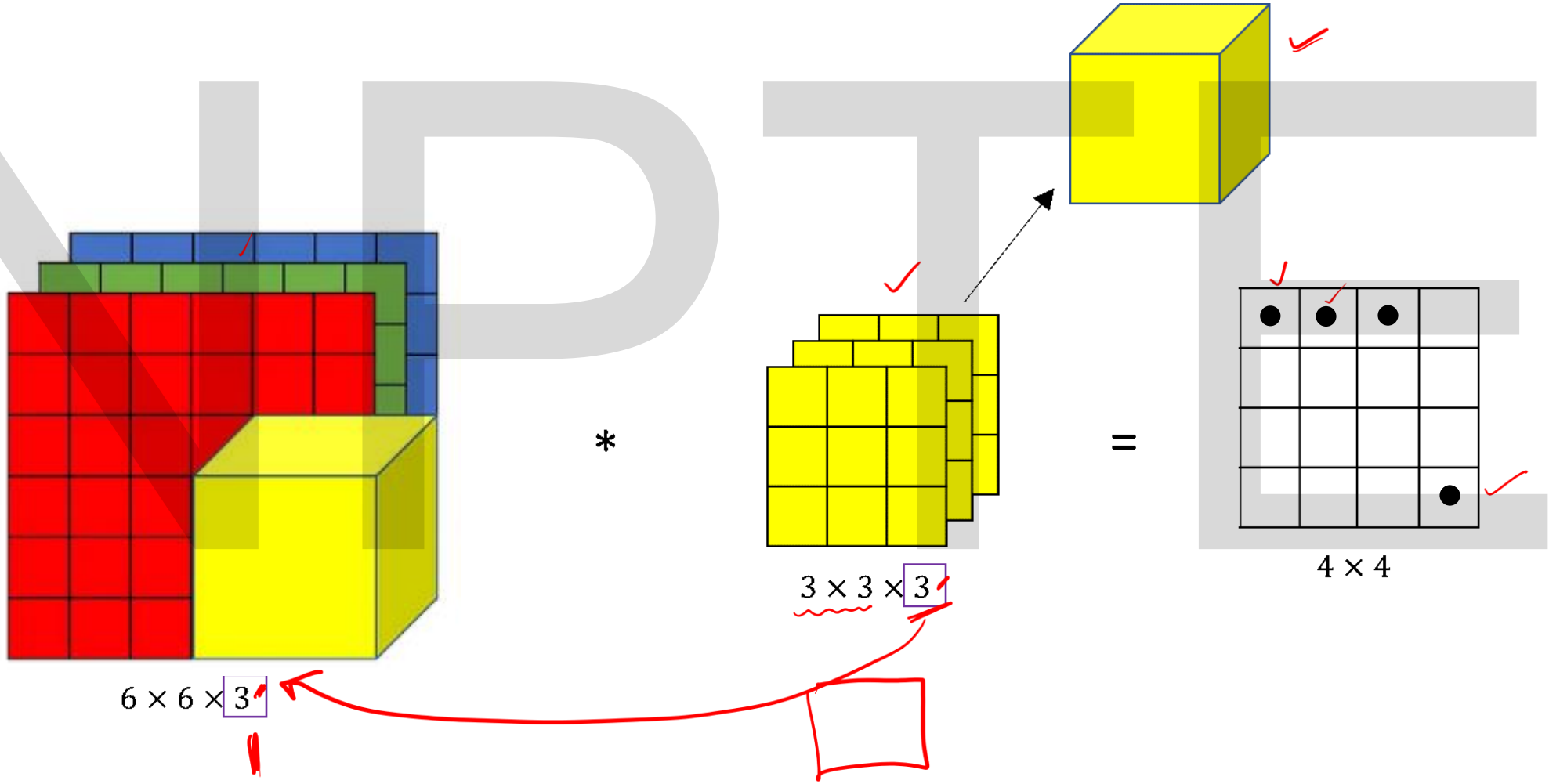
sum

21	9	-4
-11	27	23
1	8	25

feature map



Convolution operation on RGB Images



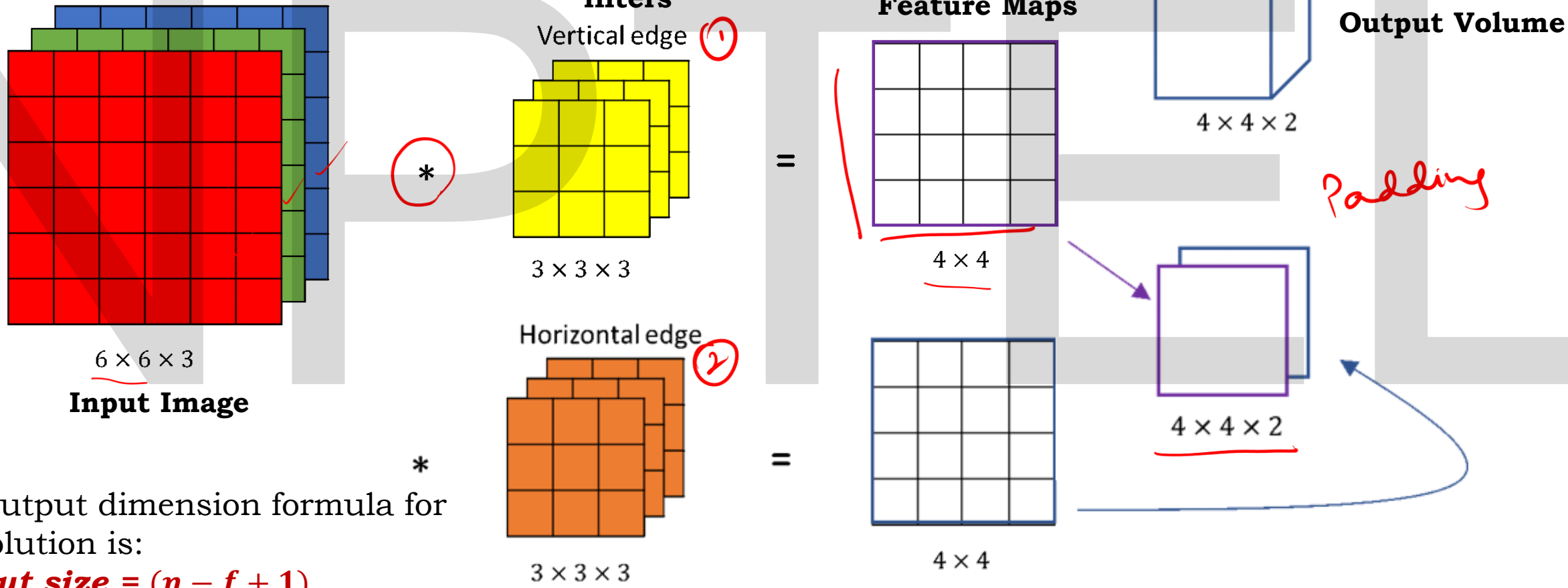
Multiple filters

$n \times n \times n_c \ * \ f \times f \times n_c \Rightarrow (n-f+1) \times (n-f+1) \times n_f$

$6 \times 6 \times 3 \ * \ 3 \times 3 \times 3 \Rightarrow (6-3+1) \times (6-3+1) \times 2$

$\Rightarrow 4 \times 4 \times 2$

Stride =1 and no padding



The output dimension formula for convolution is:

Output size $= (n - f + 1)$

For multiple filters and multiple input channels, the full representation is:

$$n \times n \times n_c \ * \ f \times f \times n_c \Rightarrow (n-f+1) \times (n-f+1) \times n_f$$

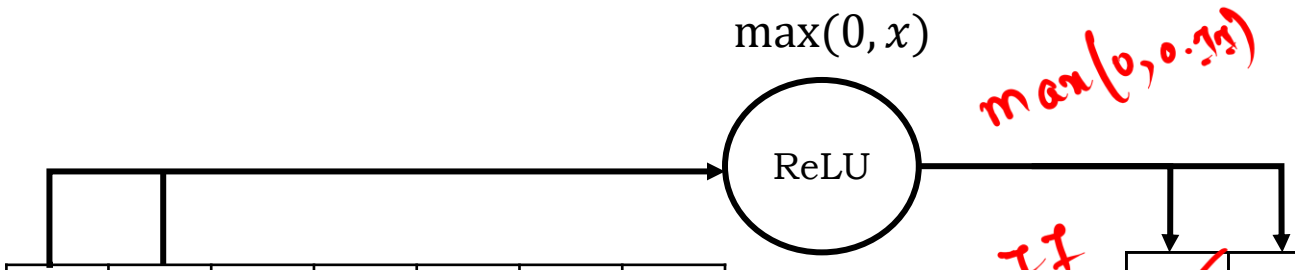
Activation Layer (ReLU)

In this layer we remove every negative values from the filtered images and replaces it with zero's.

Feature map

1	14	-9	4
-2	-20	10	6
-3	3	11	1
2	54	-2	80

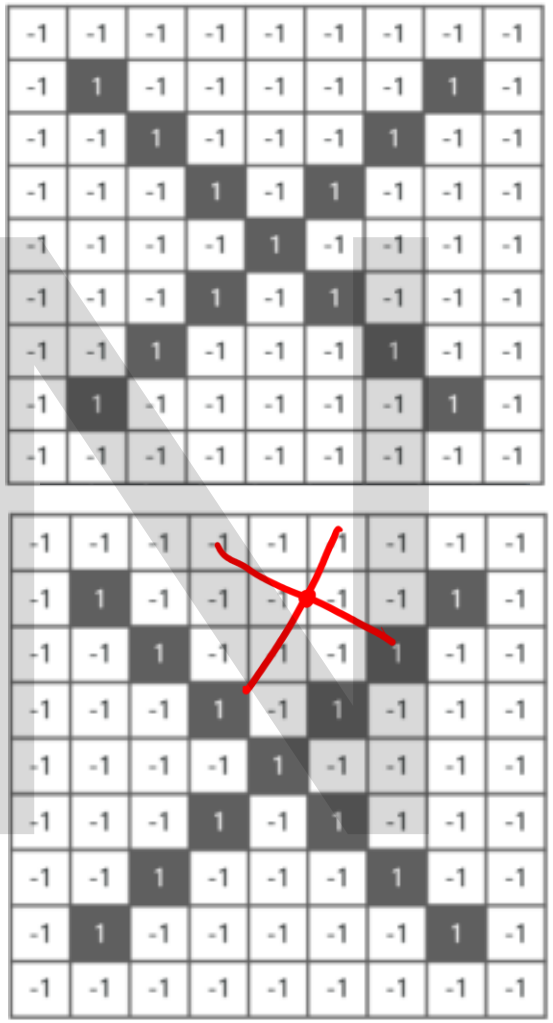
1	14	0	4
0	0	10	6
0	3	11	1
2	54	0	80



0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.00	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.00	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	0.77

ReLU

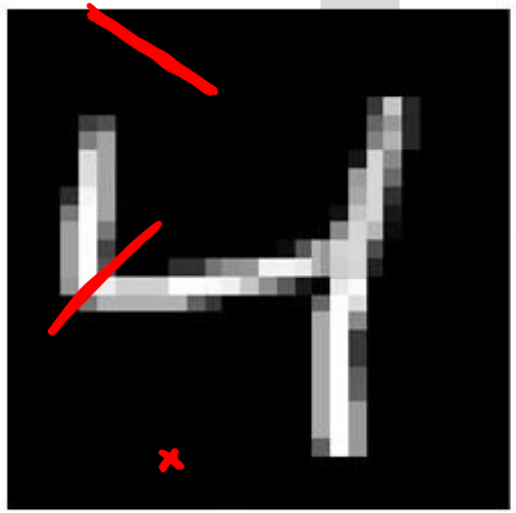


1	-1	-1
-1	1	-1
-1	-1	1

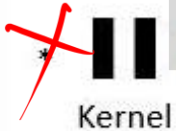
1	-1	1
-1	1	-1
1	-1	1

0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.0	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.0	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	0.77

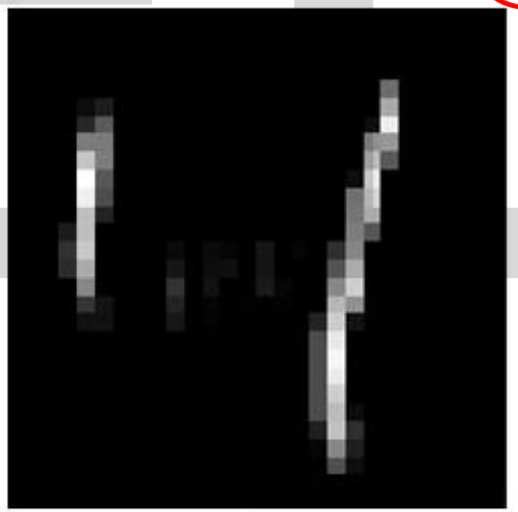


Image



Kernel

ReLU =



Output

Conv2D(32,(3,3),input_shape=(64,64,1),activation='relu')

In deep learning, parameters are the internal values that the model learns during training —the model adjusts to minimize loss.

In a convolutional layer, parameters are mainly of two types:

1.Weights (in the filters/kernels)

2. Biases (one bias per filter)

So, **parameters = weights + biases.**

The Concept of a Filter

Each filter in a CNN slides over the image and detects certain patterns (edges, corners, textures, etc.).
Every filter has small numbers inside it — these numbers are the **weights**.
If a filter is of size **5×5** and the input has **3 channels (RGB)**,
then one filter will have:

- 5 × 5 × 3 = 75 weights
  and
- +1 bias term (added to the result of the convolution).

So each filter has **76 parameters**.

For Multiple Filters

If you have **6 filters**, each filter will have its own 76 parameters.
So, total parameters = 76 × 6 = 456.

Each filter learns **different patterns** — for example:

- Filter 1 might detect vertical edges
- Filter 2 might detect horizontal edges

…and so on.

Calculating Number of Parameters

The formula is:

Parameters = $(f_h \times f_w \times n_c + 1) \times n_f$

Where:

$f_h \times f_w$: Kernel/Filter height and width

n_c : number of input channels

n_f : Number of filters

+1 bias term per filter

Let's say:

- Input: **32 × 32 × 3** (RGB image)
- Filter size: **5 × 5**
- Number of filters: **6**
- Stride = any value (doesn't affect parameter count)
- Padding = any value (doesn't affect parameter count)

Parameters = $(f_h \times f_w \times n_c + 1) \times n_f$

Parameters = (5 × 5 × 3 + 1) × 6 = 456

Layer	Filter height & width	Input Channels	# of Filters	Total Parameters
Conv1	3×3	3	64	**(3×3×3 + 1) × 64 = 1,792**
Conv2	5×5	64	128	**(5×5×64 + 1) × 128 = 204,928**

Calculating the output when an image passes through a Convolutional layer

Formula:

Output size = $\left(\frac{I-f+2P}{S}\right) + 1 \times D$

Where

I : Input

f: Filter/Kernel size

S: Stride

P: Padding

D: Number of output channels (filters), which becomes the depth of the output.

Image: 32 × 32 × 3
Filter size: 5 × 5
Number of filters: 6
Stride = 1
Padding = 0

Without padding

Output size

$= \left(\frac{I-f+2P}{S}\right) + 1 \times D$

$= \left(\frac{32-5+0}{1} + 1\right)$

$=28$

Output:

$28 \times 28 \times 6$

Image: 32 × 32 × 3
Filter size: 5 × 5
Number of filters: 6
Stride = 1
Padding = 2

With padding

Output size = $\left(\frac{I-f+2P}{S}\right) + 1 \times D$

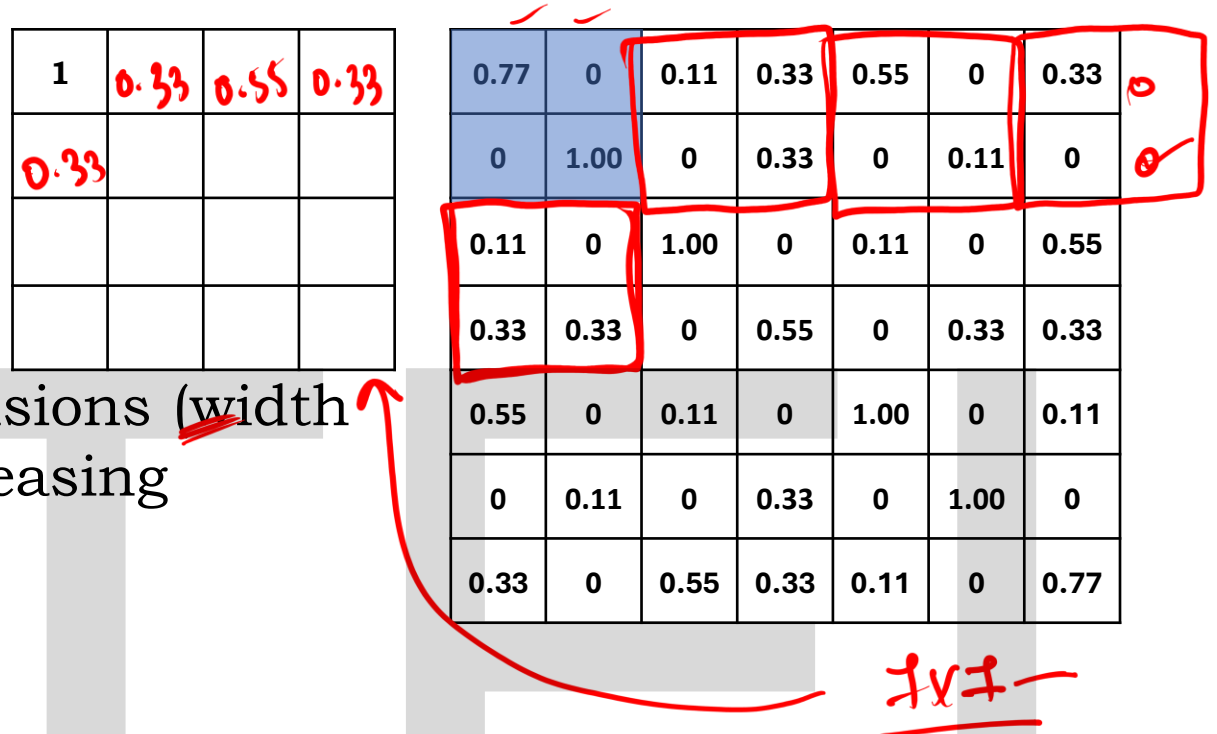
$= \left(\frac{32-5+4}{1} + 1\right)$

$=32$

Output:

$32 \times 32 \times 6$

Pooling Layer



Pooling in CNNs is a **downsampling technique** that reduces the spatial dimensions (width and height) of feature maps, thereby decreasing computational cost.

How It Works:

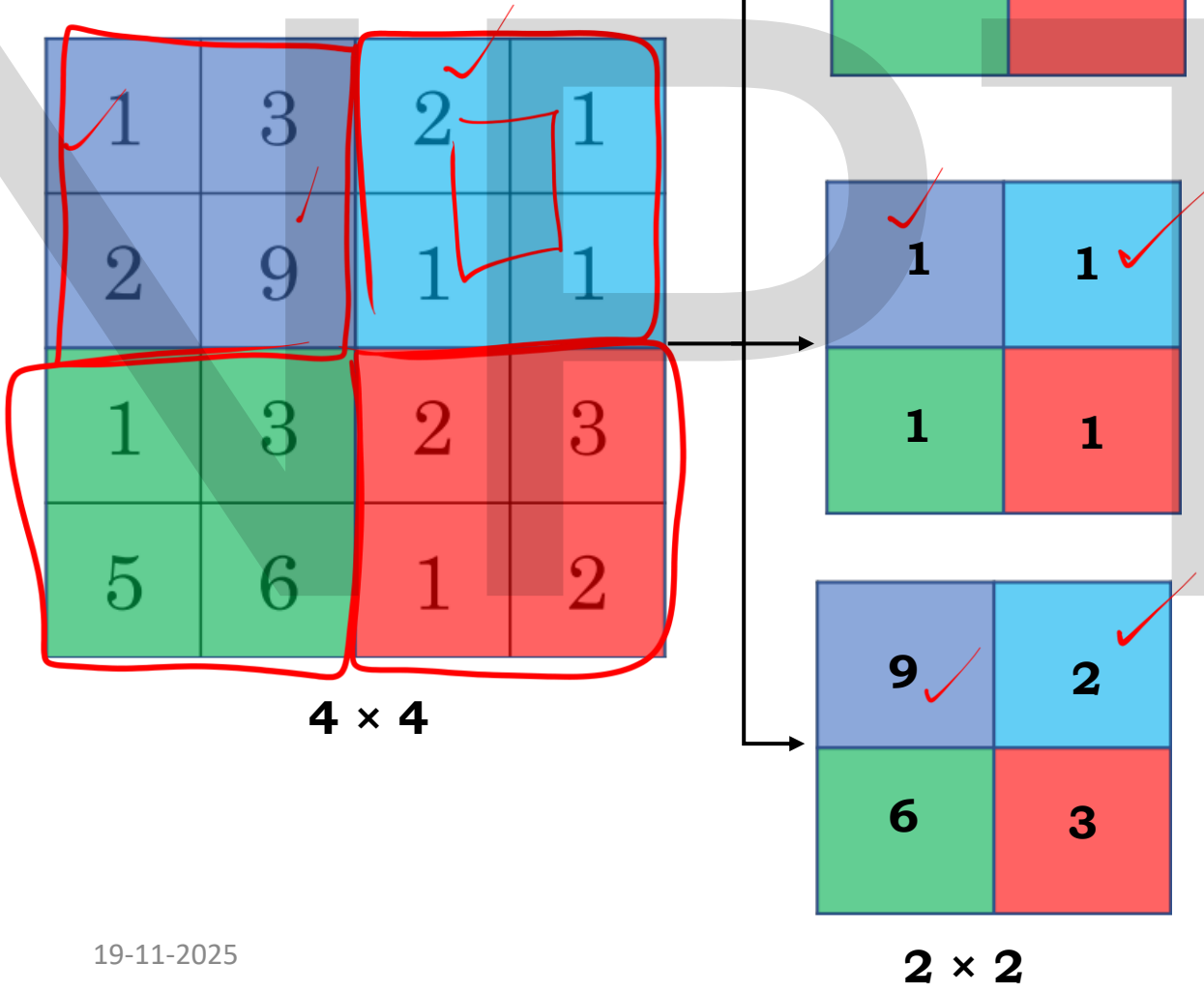
Sliding Window: A fixed-size window (e.g., 2x2) slides over the input feature map.

Summary Operation: For each window, applying the summary operation such as max pooling (taking the maximum value) or average pooling (taking the average value), to produce a smaller output.

Output Generation: The chosen value becomes a single output, creating a downsampled feature map. This process is repeated across the entire input feature map.

Types of Pooling

Stride and the window size



Average Pooling

Min Pooling

Max Pooling

0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	0.77

Max pooling

1	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

MaxPooling2D(pool_size=(2,2))

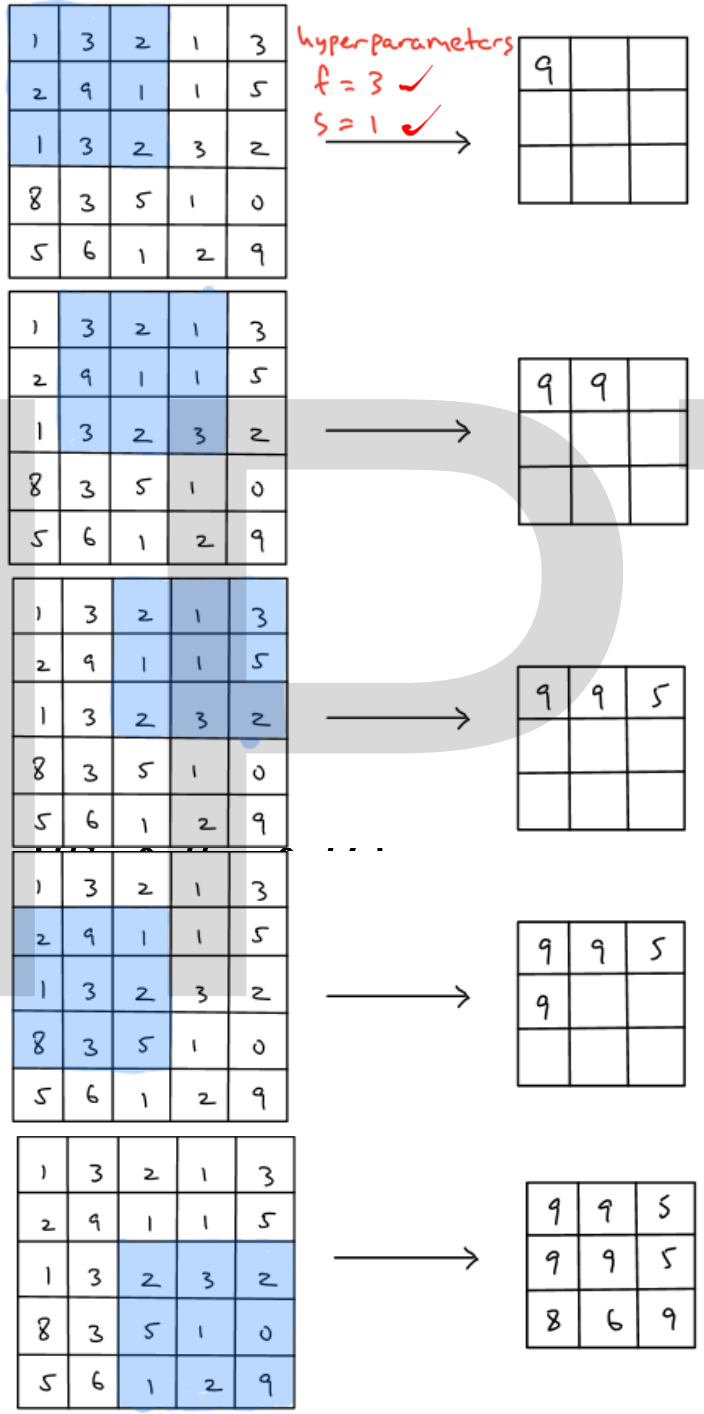
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D

model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1), padding = 'same'),
    MaxPooling2D(pool_size=(2,2), padding = 'same')
])
```

pool_size: size of the window (e.g., (2,2))
strides: step size; default = pool_size

MaxPooling2D(pool_size=(2,2), stride=(1,1))

Calcu… tput when an image passe… Pooling (Max) layer



Formula:

Output size D

Where,

I : Input to …

of conv laye…

f: Filter size

S: Stride

D: Number …er

Let's say we have:

Input: 5 × 5 × 1 (i.e., height = 5, width = 5, depth = 1)

Max Pooling:

- Filter size = 3
- Stride = 1

Apply formula to height and width:

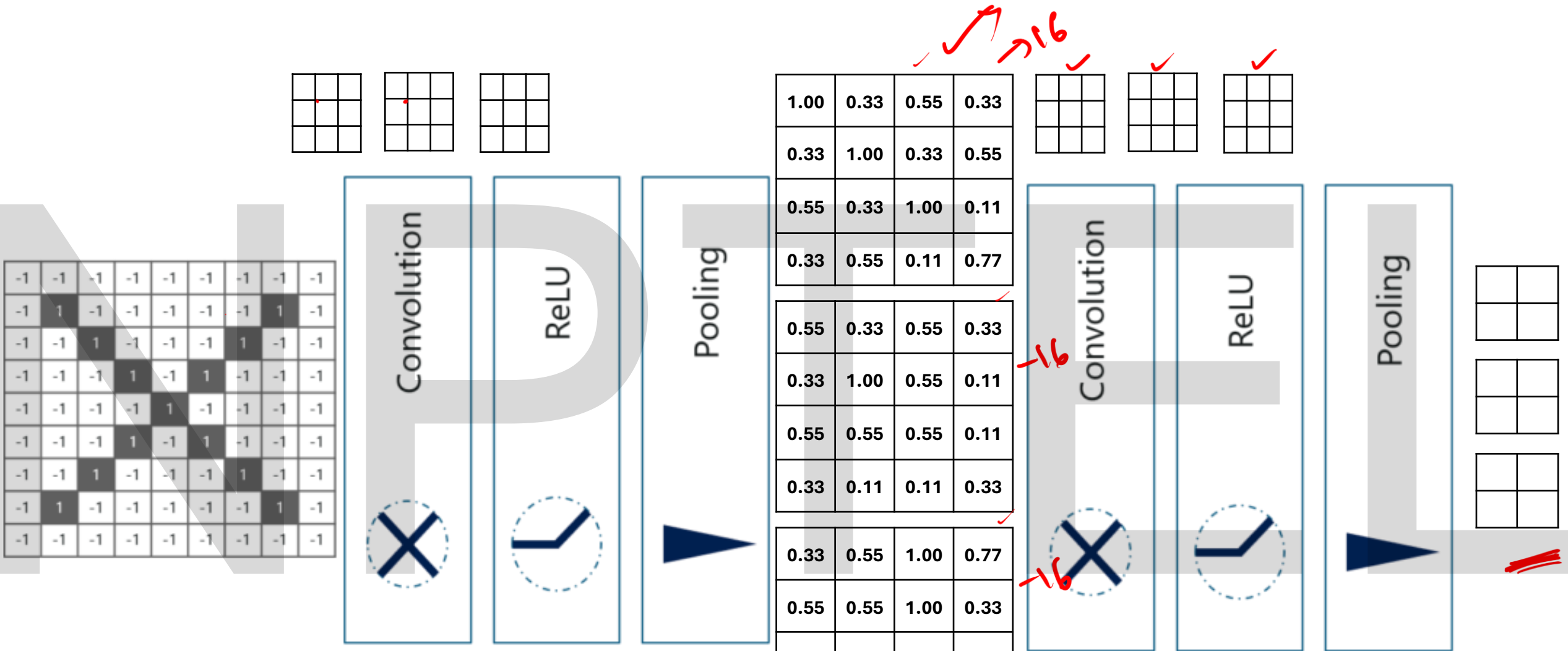
Output Height= $\left\lfloor \frac{5-3}{1} + 1 \right\rfloor = 3$

Output Width = $\left\lfloor \frac{5-3}{1} + 1 \right\rfloor = 3$

Final output = 3 × 3

If padding is applied during the conv layer, then the output size is *ceil()*

Layer Type	…er (f)	Stride (S)	Output Size
Max Pooling	… × 3	1	3 × 3 × 1



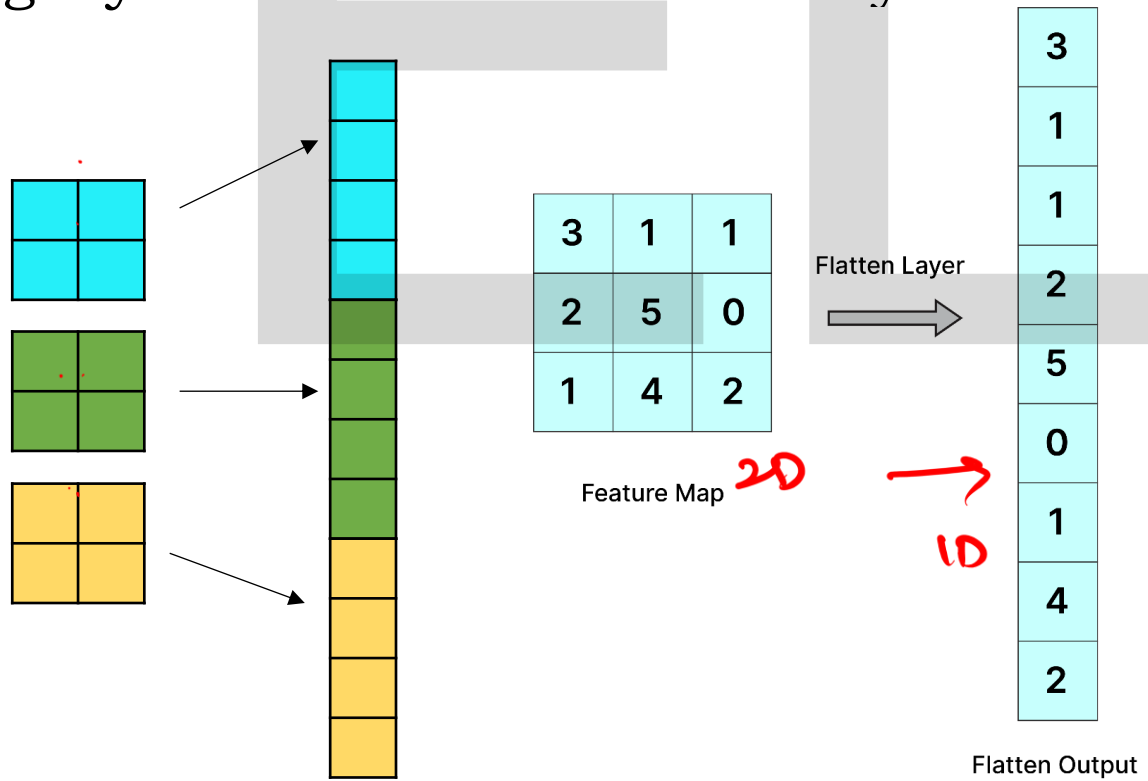

```
model = Sequential([
    Conv2D(3, (3,3), activation='relu', padding = 'same',
                input_shape=(7,7,1)),
    MaxPooling2D(pool_size=(2,2), padding='same')
])
```

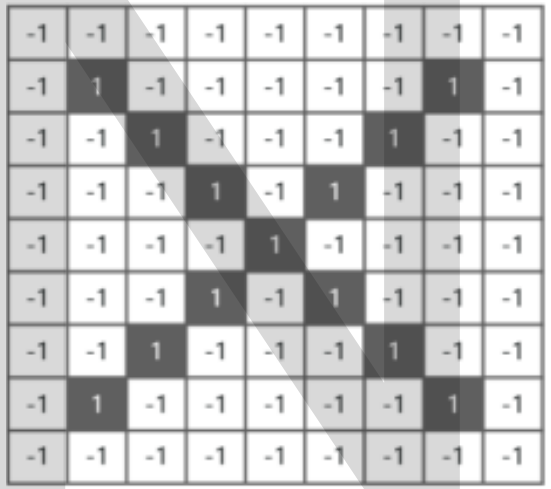
```
Conv2D(3, (3,3), activation='relu',
padding = 'same'),
MaxPooling2D(pool_size=(2,2),
padding = 'same')
```

Flatten and Fully Connected layers

- The primary purpose of the flatten layer is to **convert the multi-dimensional output of the convolutional and pooling layers into a one-dimensional vector**. This transformation is necessary **because fully connected layers**, which are **typically used for classification, require a 1D input**.
- The flatten layer is placed after the final pooling layer and before the first fully connected (dense) layer in a CNN architecture.

model.add(Flatten())





Convolution

ReLU

Pooling

1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

0.55	0.33	0.55	0.33
0.33	1.00	0.55	0.11
0.55	0.55	0.55	0.11
0.33	0.11	0.11	0.33

0.33	0.55	1.00	0.77
0.55	0.55	1.00	0.33
1.00	1.00	0.11	0.55
0.77	0.33	0.55	0.33

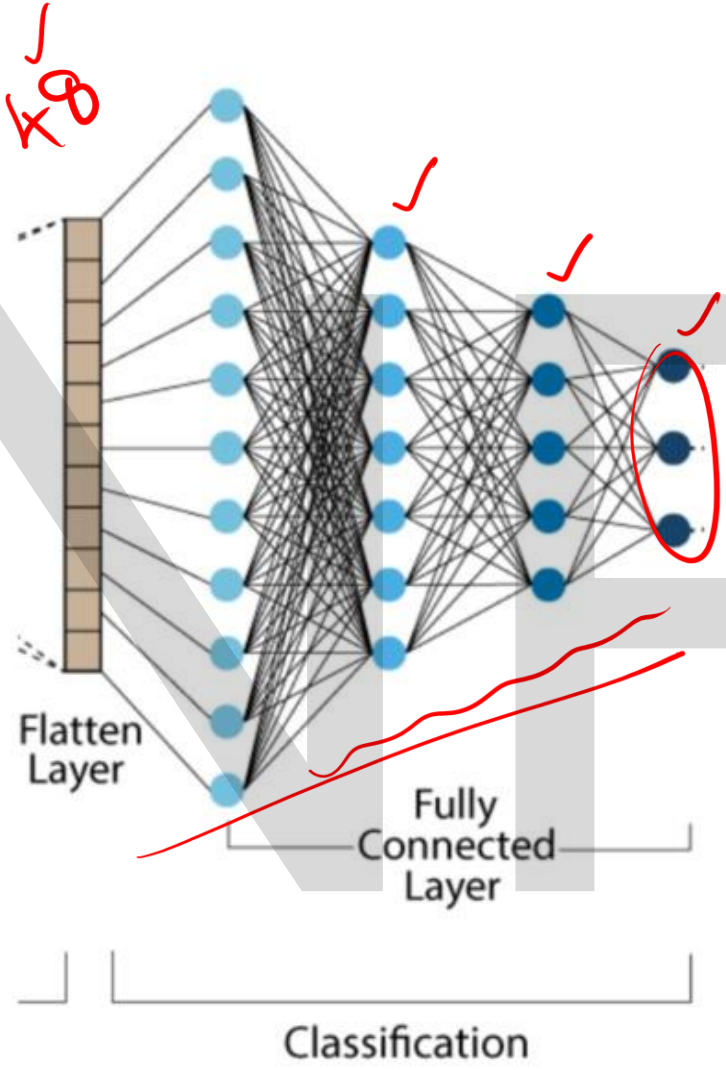
48 neurons

Convolution

ReLU

Pooling

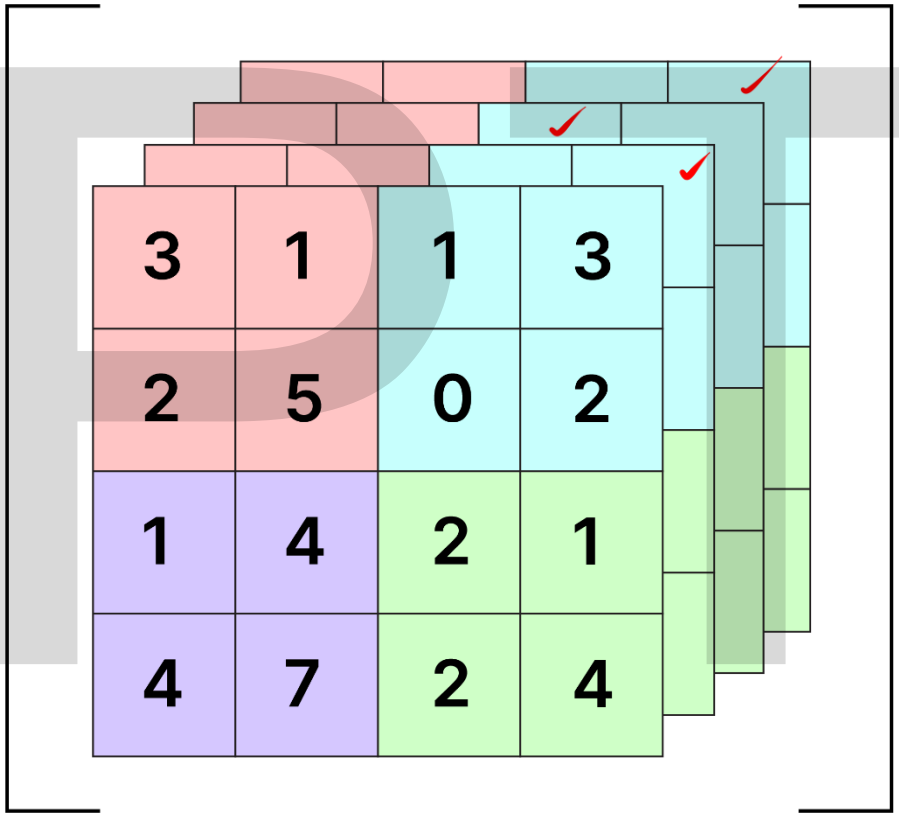
12 neurons



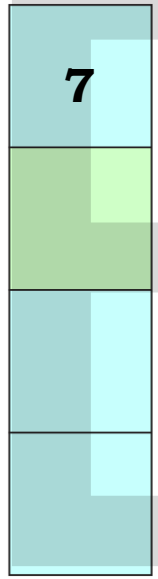

```
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(units=3,activation='softmax'))
model.compile(optimizer='adam',loss='categorical_cross entropy',
metrics=['accuracy'])
model.fit(training_set, epochs = 10)
```

GlobalMaxPooling & GlobalAveragePooling

Global Max Pooling



=




```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, GlobalAveragePooling2D, Dense

model = Sequential([
    # First conv block
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D(pool_size=(2,2)),

    # Second conv block
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(pool_size=(2,2)),

    # Global Average Pooling instead of Flatten
    GlobalAveragePooling2D(),

    # Dense layers
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])
model.summary()
```

Summary

- We learned **how multiple filters capture** different spatial features and how CNNs handle RGB inputs efficiently.
- we extended our understanding of CNNs by exploring how convolutional outputs are processed through **activation, pooling, and flatten layers to prepare for final classification.**

Next session

In the upcoming session, you will:

- Understand **overfitting in CNNs** and explore techniques to prevent it from practical perspective:
  - **Dropout, Batch Normalization, and Data Augmentation.**

Thank you